Reviewer Pack — GCT Lie-Stabilizer Codimension Linearly Bounds ACC⁰ Size

Entry 83d26c2ef069 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 00:01:32 UTC

GCT Lie-Stabilizer Codimension Linearly Bounds ACC⁰ Size

Entry ID: `83d26c2ef069` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 00:01:32 UTC

1. Conjecture

Field A (mathematical branch): Geometric Complexity Theory: Lie-algebra stabilizers of multilinear polynomial extensions in the Mulmuley-Sohoni-Bürgisser-Ikenmeyer framework **Field B** (complexity object): ACC⁰[m] circuit size for Boolean functions (Sipser-like targets)

Statement:

> For a Boolean function f:{-1,+1}^n -> {-1,+1}, let F_f(x) = Σ_{S⊆[n]} f̂(S) · Π_{i∈S} x_i be its multilinear Walsh extension over Q, let L(F_f) := {A ∈ gl_n(Q) : Σ_{i,j} A_{ij} x_j ∂_i F_f ≡ 0 in Q[x_1,...,x_n]} be its GCT Lie stabilizer, and set γ(f) := n^2 − dim_Q L(F_f). Conjecture: there is an absolute constant C ≤ 10 such that every ACC⁰[m] circuit of size s computing f satisfies γ(f) ≥ n^2 − C·s. A single ACC⁰[m] circuit of size s computing some f with γ(f) < n^2 − 10·s falsifies the conjecture.

Rationale (proposer's reasoning):

> GCT's symmetry-characterization principle says polynomials whose stabilizer Lie algebras exceed generic dimension are atypical; we invert this to claim small ACC⁰ circuits cannot accidentally synthesize multilinear extensions of anomalously high symmetry. The Lie stabilizer is GL_n-equivariant rather than truth-table-dense, so it dodges Razborov–Rudich's 'large + constructive' clause, and it is computed from the GL_n action on polynomials, not from black-box algebraic extensions — placing the invariant outside the algebrization and relativization regimes that GCT was designed to escape.

Taxonomy category: `GCT` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `28e89ae558787cfc`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across n∈{4,5,6,7} and s∈{3,8,20,50}, with 30 seeds per (n,s), compute slack_t := γ(f_t) − (n² − 10·s) for every trial t. SUPPORTED iff min slack ≥ 0 over all 480 trials AND on reference functions (parity, AND, MAJ, Sipser) slack ≥ 0. FALSIFIED iff any single trial yields slack < 0.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric complexity theory Lie algebra stabilizer Boolean function circuit lower bound` - `Walsh Fourier multilinear extension symmetry Lie stabilizer ACC0 circuit complexity` - `Mulmuley Sohoni stabilizer dimension ACC0 lower bound polynomial`

Top relevant hits considered: - [http://arxiv.org/abs/1212.5380v2] On properties of principal elements of Frobenius Lie algebras - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2411.11095v3] Invariant theory and coefficient algebras of Lie algebras - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1504.04131v2] Formulas for the Walsh coefficients of smooth functions and their application to bounds on the Walsh coefficients - [http://arxiv.org/abs/2106.02913v1] Projective collineations of decomposable spacetimes generated by the Lie point symmetries of geodesic equations - [http://arxiv.org/abs/0704.0213v2] Geometric Complexity Theory V: On deciding nonvanishing of a generalized Littlewood-Richardson coefficient - [http://arxiv.org/abs/cs/0703110v4] Geometric Complexity Theory IV: nonstandard quantum group for the Kronecker problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random
import math
import itertools
from fractions import Fraction

def generate_random_acc0_circuit(n, s, m=6):
    if s < 3:
        return [['AND', [('AND', [i]) for i in range(n)]]]
    circuit = []
    for _ in range(s):
        if random.random() < 0.5:
            inputs = random.sample(range(n), random.randint(1, n))
            circuit.append(['AND', [('AND', [i]) for i in inputs]])
        else:
            inputs = random.sample(range(n), random.randint(1, n))
            circuit.append(['MOD', m, [('AND', [i]) for i in inputs]])
    return circuit
```

```
def evaluate_circuit(circuit, x):
    if isinstance(circuit, list):
        if circuit[0] == 'AND':
            return all(evaluate_circuit(sub, x) for sub in circuit[1])
        elif circuit[0] == 'MOD':
            m = circuit[1]
            return sum(evaluate_circuit(sub, x) for sub in circuit[2]) % m == 0
    elif isinstance(circuit, tuple):
        if circuit[0] == 'AND':
            return x[circuit[1]]
    return False

def walsh_hadamard_transform(circuit, n):
    f_hat = {}
    for alpha in itertools.product([-1, 1], repeat=n):
        f_hat[alpha] = sum(evaluate_circuit(circuit, x) * math.prod(alpha[i] * x[i] for i in range
                           for x in itertools.product([-1, 1], repeat=n))
    return f_hat

def build_linear_system(f_hat, n):
    system = []
    for alpha in itertools.product([-1, 1], repeat=n):
        row = []
        for i in range(n):
            for j in range(n):
                if i == j:
                    row.append(Fraction(f_hat[alpha] * alpha[i], 1))
                else:
                    row.append(Fraction(0, 1))
        system.append(row)
    return system

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        if matrix[i][i] == 0:
            for j in range(i+1, n):
                if matrix[j][i] != 0:
                    matrix[i], matrix[j] = matrix[j], matrix[i]
                    break
        if matrix[i][i] == 0:
            continue
        for j in range(i+1, n):
            factor = matrix[j][i] / matrix[i][i]
            for k in range(i, n):
                matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for i in range(n) if any(matrix[i][j] != 0 for j in range(n)))
    return n - rank

def run_trial(seed):
    random.seed(seed)
    n = random.choice([4, 5, 6, 7])
    s = random.choice([3, 8, 20, 50])
    circuit = generate_random_acc0_circuit(n, s)
    f_hat = walsh_hadamard_transform(circuit, n)
```

```
    system = build_linear_system(f_hat, n)
    gamma = gaussian_elimination(system)
    slack = gamma - (n**2 - 10*s)
    conjecture_holds = slack >= 0
    counterexample = f"n={n}, s={s}, circuit={circuit}" if not conjecture_holds else ""
    return {
        "metric_name": "slack",
        "metric_value": slack,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = seeds[next(i for i, r in enumerate(results) if not r["conjecture_hold
        counterexample = results[next(i for i, r in enumerate(results) if not r["conjecture_holds'
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_698adb63.py", line 102, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_698adb63.py", line 86, in run_trial
    gamma = gaussian_elimination(system)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_698adb63.py", line 65, in gaussian_elim
    if matrix[i][i] == 0:
       ~~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with an IndexError in gaussian_elimination before any of the 480 trials or reference-function checks produced slack data, so the pre-registered support condition cannot be evaluated. Per Rule 2, a crash yields INCONCLUSIVE. | next: Fix the Gaussian elimination routine (guard against out-of-range pivot indexing and handle rectangular/short matrices) and rerun the full (n,s)-grid plus reference-function suite to obtain actual slack statistics.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	325259
2	preregistration	claude_max	opus	0	0	6562
3	novelty	claude_max	opus	0	0	3968
4	novelty	claude_max	opus	0	0	8726
5	test_gen	mistral	codestral-latest	0	0	309881
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	294528
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	259141
8	test_gen	mistral	codestral-latest	0	0	309995
9	judge	claude_max	opus	0	0	4571

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1522632 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 2, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/83d26c2ef069.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/83d26c2ef069.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/83d26c2ef069.tar.gz` (if generated)